Rules for Evaluating / Parsing Expressions in Java

Java has well-defined rules for specifying the order in which the operators in an expression are evaluated when the expression has several operators. For example, multiplication and division have a higher precedence than addition and subtraction. Precedence rules can be overridden by explicit parentheses.

Precedence order.

When two operators share an operand the operator with the higher *precedence* goes first. For example, 1 + 2 * 3 is treated as 1 + (2 * 3), whereas 1 * 2 + 3 is treated as (1 * 2) + 3 since multiplication has a higher precedence than addition.

Associativity.

When an expression has two operators with the same precedence, the expression is evaluated according to its *associativity*. For example `x = y = z = 17` is treated as `x = (y = (z = 17))`, leaving all three variables with he value 17, since the `=` operator has right-to-left associativity (and an assignment statement evaluates to the value n the right hand side). On the other hand, `72 / 2 / 3` is treated as `(72 / 2) / 3` since the `/` operator has left-to-right associativity. Some operators are not associative: for example, the expressions `(x <= y <= z)` and `++x--` are invalid.

Precedence and associativity of Java operators.

The table on the next page shows all Java operators from highest to lowest precedence, along with their associativity. Most programmers do not memorize them all, and even those that do still use parentheses for clarity.

There is no explicit operator precedence table in the Java Language Specification. Different tables on the web and in textbooks disagree in some minor ways.

Order of evaluation of subexpressions.

Associativity and precedence determine in which order Java applies operators to subexpressions but they do not determine in which order the subexpressions are evaluated. In Java, subexpressions are evaluated from left to right (when there is a choice). So, for example in the expression `A() + B() * C(D(), E())`, the subexpressions are evaluated in the order `A()`, `B()`, `D()`, `E()`, and `C()`. Although, `C()` appears to the left of both `D()` and `E()`, we need the results of both `D()` and `E()` to evaluate `C()`. It is considered poor style to write code that relies upon this behavior (and different programming languages may use different rules).

Also, if i=5, **i = i / ++i** results in i=0 because the operands are evaluated from left to right first and then the operators:

$i_{(1)}$ $=_{(6)}$ $i_{(2)}$ $/_{(5)}$ $++_{(4)}$ $i_{(3)}$ Thus the division is 5/6, which results in zero.

Short circuiting. When using the conditional *and* and *or* operators (`&&` and `||`), Java does not evaluate the second operand unless it is necessary to resolve the result. This allows statements like `if (s != null && s.length() < 10)` to work reliably. Programmers rarely use the non short-circuiting versions (`&` and `|`) with `boolean` expressions.

Special Cases. In the expression **`new Class().`**`func()` the constructor is called and then the method, whereas in `new` **`Outer.Inner()`** the outer class is identified first and then the inner constructor is called. In the first case the new operator runs first, but in the second the member accessor operator (dot) runs first. This is due to the rules of Java syntax, not the rules for operator precedence and associativity.